

Архитектура Express-приложения

Routes, Controllers и Services

Результаты занятия

- ✓ Объяснять назначение routes, controllers и services
- ✓ Разделять маршрутизацию, HTTP-логику и бизнес-логику
- ✓ Выносить обработчики маршрутов в controllers
- ✓ Выносить операции с данными и правила приложения в services
- ✓ Передавать данные между route, controller и service
- ✓ Организовывать структуру Express-проекта по слоям
- ✓ Понимать путь HTTP-запроса через архитектурные компоненты

Учебные вопросы

Зачем разделять приложение на слои

Проблема смешивания маршрутизации, HTTP и бизнес-логики

Controllers

Получение данных из req и формирование res

Взаимодействие слоёв

Полный путь запроса и структура проекта

Routes

Endpoints и связь HTTP-запроса с controller

Services

Бизнес-логика и работа с данными независимо от HTTP



Учебный вопрос 1. Зачем разделять приложение на слои

От работающего кода к понятной архитектуре

Express не навязывает архитектуру проекта

- Express предоставляет маршрутизацию, middleware и работу с HTTP-запросами и ответами.
- Фреймворк не требует создавать папки controllers или services.
- Разделение на routes, controllers и services является архитектурным соглашением разработчиков.
- Такой подход становится полезным, когда приложение начинает расти.
- Главная цель — разделить разные обязанности между разными модулями.

Главная идея

Routes, controllers и services — архитектурные роли, а не специальные классы Express.

Когда вся логика находится в маршруте

```
router.get('/:id', (req, res) => {  
  const id = Number(req.params.id);  
  const product = products.find(p => p.id === id);  
  
  if (!product) {  
    return res.status(404).json({  
      message: 'Product not found'  
    });  
  }  
  
  res.json(product);  
});
```

Что смешано в одном обработчике

- Описание HTTP-маршрута GET `/:id`.
- Получение `id` из `req.params`.
- Преобразование значения в `Number`.
- Поиск товара в данных.
- Проверка результата поиска.
- Выбор HTTP-статуса.
- Формирование JSON-ответа.

Главная идея

Для небольшого примера это допустимо, но по мере роста обработчики становятся перегруженными.

От толстого маршрута к трём слоям

Один маршрут может многое. Но лучше, когда у каждого своя ответственность.

Один большой маршрут

✗ Всё в одном месте

```
1 // Получение товара по id (толстый маршрут)
2 router.get('/products/:id', async (req, res) => {
3
4   // 1. Чтение данных из запроса
5   const { id } = req.params;
6   const { userId } = req.query;
7
8   // 2. Бизнес-проверки / валидация
9   if (!id || isNaN(id)) {
10     return res.status(400).json({
11       message: 'Некорректный id товара'
12     });
13   }
14
15   // 3. Работа с данными (поиск в БД)
16   const product = await Product.findById(id);
17   if (!product) {
18     return res.status(404).json({
19       message: 'Товар не найден'
20     });
21   }
22
23   // 4. Формирование и отправка ответа
24   return res.json({
25     success: true,
26     data: product
27   });
28
29 });
```

Чтение req
(req.params,
req.body, req.query)

Бизнес-логика
и валидация

Поиск / изменение
данных

Отправка ответа
(res.json())

Разделение ответственности

Выделяем разные
задачи в отдельные
слои



Из одного большого
маршрута
→ в три понятных слоя

✦ Три слоя — чёткие обязанности, понятный код, проще развивать

1 Route Маршрут (роутер)

```
// routes/products.js
const express = require('express');
const router = express.Router();

const { getProductById } =
  require('../controllers/
    products.controller');

router.get('/products/:id',
  getProductById);

module.exports = router;
```

Отвечает за:

- URL (путь)
- HTTP-метод (GET, POST и т.д.)
- подключение controller
- может содержать middleware (например, проверку авторизации)

Минимум логики.
Только маршрутизация.

2 Controller Контроллер

```
// controllers/products.controller.js
const productService =
  require('../services/
    products.service');

async function getProductById
  (req, res) {
  const { id } = req.params;
  try {
    const product = await
      productService.getById(id);
    return res.json({
      success: true,
      data: product
    });
  } catch (error) {
    return res.status(500).json({
      message: error.message
    });
  }
}

module.exports = { getProductById };
```

Отвечает за:

- читает req (req.params, req.body и т.д.)
- вызывает service
- обрабатывает ошибки
- выбирает HTTP-статус код
- формирует и отправляет HTTP-ответ (res.json(), res.status())

3 Service Сервис (бизнес-логика)

```
// services/products.service.js
const Product = require('../models/
  product.model');

async function getById(id) {
  // Валидация
  if (!id || isNaN(id)) {
    throw new Error('Некорректный
      id товара');
  }
  // Поиск данных
  const product = await
    Product.findById(id);
  if (!product) {
    throw new Error('Товар не найден');
  }
  // Дополнительная бизнес-логика
  return product;
}

module.exports = { getById };
```

Отвечает за:

- поиск / изменение данных (работа с БД)
- бизнес-логика (правила приложения)
- валидация
- может вызывать внешние сервисы (API, очереди и т.д.)

Результат (res.json()) возвращается в Controller



Функциональность не меняется, но код становится понятнее и удобнее для сопровождения.



Чистая архитектура
— залог роста проекта!

Разделяем три ответственности

Routes

Определяют HTTP-метод, URL и функцию, которой нужно передать запрос.

Controllers

Работают с req и res: получают данные запроса и формируют HTTP-ответ.

Services

Выполняют бизнес-операции и не должны зависеть от объектов req и res.

Кто за что отвечает

Слой	Основная ответственность	Знает о HTTP
Route	Какой endpoint вызывает какую функцию	Да
Controller	Получение запроса и отправка ответа	Да
Service	Бизнес-логика и операции с данными	Нет

Архитектура Route → Controller → Service

Как HTTP-запрос проходит через слои Express-приложения



HTTP-слой — обработка запроса и формирование ответа

Клиент, маршруты, контроллеры, HTTP-объекты (req, res)

1

Клиент

Браузер / Frontend



Отправляет
HTTP-запрос

GET /api/products/15

HTTP-запрос

GET
/api/products/15

2

Route

Маршрут



Определяет endpoint
и передаёт управление
в контроллер

```
router.get('/:id',  
  getProductById);
```

передаёт
req, res

3

Controller

Контроллер



Получает req,
извлекает данные
и вызывает сервис

```
const { id } = req.params;  
const product = await  
  productService.getById(id);
```

req.params / req.query /
req.body



Бизнес-слой — логика приложения

Сервисы, работа с данными, бизнес-правила

4

Service

Сервис



Выполняет
бизнес-операцию:
работа с базой данных,
бизнес-правила, валидация

```
const product = await  
  productRepository.findById(id);  
return product;
```

вызов
service

результат
возвращает
данные

6

Клиент

Получает ответ



Получает HTTP-ответ
и отображает данные

```
{  
  "success": true,  
  "data": { ... }  
}
```

HTTP-ответ

res.json(...)

5

Controller

Формирует ответ

Получает результат от сервиса
и отправляет HTTP-ответ

```
return res.json({  
  success: true,  
  data: product  
});
```



Примеры бизнес-операций

- ✓ Поиск и сохранение данных
- ✓ Применение бизнес-правил
- ✓ Валидация
- ✓ Работа с внешними сервисами (платежи, уведомления и т.д.)



HTTP-слой отвечает за запрос и ответ, бизнес-слой — за логику приложения.

Такое разделение делает код понятнее, проще в поддержке и удобнее для тестирования.



Чистая архитектура
— залог масштабируемого
приложения!



Учебный вопрос 2. Routes

Маршрут знает URL, HTTP-метод и нужный controller

Роль routes

- Route описывает доступный endpoint приложения.
- Он связывает HTTP-метод и URL с функцией-обработчиком.
- После разделения архитектуры обработчиком обычно становится функция controller.
- Route не должен содержать большой объём бизнес-логики.
- Файл routes удобно рассматривать как карту API конкретного ресурса.

Route без бизнес-логики

```
router.get('/', getProducts);  
router.get('/:id', getProductById);  
router.post('/', createProduct);  
router.patch('/:id', updateProduct);  
router.delete('/:id', deleteProduct);
```

Что важно

- Route показывает структуру API.
- GET / связан с getProducts.
- GET /:id связан с getProductById.
- Маршрут не знает, как именно выполняется поиск или изменение товара.
- Подробная логика вынесена из файла маршрутизации.

Перегруженный и компактный route

Перегруженный

- Получает req.params
- Ищет данные
- Проверяет правила
- Изменяет массив
- Формирует ответ

Компактный

- Определяет HTTP-метод
- Определяет URL
- Подключает middleware
- Указывает controller
- Легко показывает структуру API

productRoutes.js

```
import express from 'express';
import {
  getProducts,
  getProductById,
  createProduct,
  updateProduct,
  deleteProduct
} from '../controllers/productController.js';

const router = express.Router();

router.get('/', getProducts);
router.get('/:id', getProductById);
router.post('/', createProduct);
router.patch('/:id', updateProduct);
router.delete('/:id', deleteProduct);
export default router;
```

Route остаётся частью Express

- Routes создаются через `express.Router()`.
- Маршруты по-прежнему используют HTTP-методы Express.
- Можно подключать route-level middleware.
- Можно использовать параметры пути.
- Изменяется не механизм Express, а организация собственного кода.



Учебный вопрос 3. Controllers

Контроллер связывает HTTP-запрос с логикой приложения

Роль controller

- Controller — функция обработки HTTP-запроса, которая получает данные из req, вызывает необходимую логику приложения и формирует ответ через res.
- Controller знает о req и res.
- Он получает params, query и body.
- Он вызывает функции service.
- Controller выбирает подходящий HTTP-статус.
- Он формирует JSON-ответ для клиента.

Простой controller

```
export function getProducts(req, res) {  
  res.json(products);  
}
```

Что важно

- Функция экспортируется из отдельного файла.
- Route может передать ей req и res автоматически.
- Пока controller содержит работу с данными напрямую.
- Следующим шагом эту операцию перенесём в service.

Controller получает id из запроса

```
export function getProductById(req, res) {  
  const id = Number(req.params.id);  
  const product = findProductById(id);  
  
  if (!product) {  
    return res.status(404).json({  
      message: 'Product not found'  
    });  
  }  
  
  res.json(product);  
}
```

Что важно

- Controller читает req.params.id.
- Значение преобразуется в число.
- Поиск передаётся функции findProductById.
- Controller интерпретирует результат и формирует HTTP-ответ.

Что логично оставить в controller

Задача	Пример
Чтение route params	<code>req.params.id</code>
Чтение query	<code>req.query.category</code>
Чтение body	<code>req.body</code>
Выбор HTTP-статуса	<code>res.status(201)</code>
Формирование ответа	<code>res.json(result)</code>

Controller не должен становиться новым app.js

- Простое перемещение всего кода из route в controller не решает архитектурную проблему.
- Если controller содержит большие вычисления и правила, он снова становится перегруженным.
- Бизнес-операции лучше вынести ещё на один уровень.
- Для этого используется service layer.

Главная идея

Controller должен координировать HTTP-запрос, а не содержать всю бизнес-логику приложения.

Controller как граница между HTTP и бизнес-логикой

Как controller связывает HTTP-запрос с service



HTTP-мир

Данные приходят в HTTP-запросе

Клиент (браузер, мобильное приложение и т.д.) отправляет запрос на ваш сервер

/id

req.params

Параметры маршрута

```
{ id: "15" }
```



req.query

Строка запроса (query-параметры)

```
{ page: "1", limit: "10",  
  sort: "price" }
```



req.body

Тело запроса (JSON)

```
{ name: "Ноутбук",  
  price: 1000 }
```



res.status(...).json(...)

Отправляет HTTP-ответ клиенту

```
{  
  success: true,  
  data: { ... }  
}
```



Controller

Граница между HTTP и бизнес-логикой

Читает данные из req, вызывает service и формирует HTTP-ответ

- 1 читает req.params
- 2 читает req.query
- 3 читает req.body
- 4 вызывает service (передает обычные JS-данные)
- 5 формирует res.status(...).json(...)

// Пример controller

```
const getProduct = async (req, res) => {  
  const id = Number(req.params.id);  
  const filters = req.query;  
  const data = req.body;
```

```
  const result = await productService.getById(id,  
    filters, data);
```

```
  return res.status(200).json(result);  
};
```

обычные
JS-данные

```
{  
  id: 15,  
  filters:{...},  
  data: {...}  
}
```

результат
(обычные данные)



Бизнес-логика (Service)

Работает с обычными данными

Содержит бизнес-правила, операции с данными, валидацию и т.д.



ProductService

Получает обычные параметры и выполняет бизнес-операции

```
class ProductService {  
  async getById(id, filters, data) {  
    // Бизнес-логика: поиск, проверки,  
    // работа с базой данных и т.д.  
    const product = await db.product.findById(id);  
    return product;  
  }  
}
```



Service не получает req и res

Service работает только с обычными JS-данными и не зависит от HTTP (его легче тестировать и переиспользовать).

```
{  
  id: 15,  
  name: "Ноутбук",  
  price: 1000  
}
```



Controller — граница между HTTP и бизнес-логикой: он переводит данные запроса в обычные аргументы service и превращает результат service в HTTP-ответ.



Чистое разделение делает код понятнее, проще тестировать и развивать приложение.



Учебный вопрос 4. Services

Выносим бизнес-операции из HTTP-слоя

Что такое service

- Service — модуль с функциями бизнес-логики, которые выполняют операции приложения и не должны зависеть от Express req и res.
- Service получает обычные JavaScript-значения.
- Он может искать, создавать или изменять данные.
- Service возвращает результат controller.
- Он не должен формировать HTTP-ответ.
- Позже service сможет работать с PostgreSQL вместо массива.

Controller и Service

Controller

- Получает req и res
- Работает с HTTP
- Извлекает входные данные
- Выбирает HTTP-статус
- Отправляет ответ

Service

- Получает обычные аргументы
- Не зависит от HTTP
- Выполняет бизнес-операцию
- Работает с данными
- Возвращает результат

Service для получения товаров

```
export function getAllProducts() {  
  return products;  
}  
  
export function findProductById(id) {  
  return products.find(  
    product => product.id === id  
  );  
}
```

Что важно

- Функции не получают req и res.
- На вход findProductById поступает обычное число id.
- Service выполняет операцию поиска.
- Результат возвращается вызывающему коду.

Controller вызывает Service

```
export function getProductById(req, res) {  
  const id = Number(req.params.id);  
  const product = findProductById(id);  
  
  if (!product) {  
    return res.status(404).json({  
      message: 'Product not found'  
    });  
  }  
  
  res.json(product);  
}
```

Что важно

- Controller получает id из HTTP-запроса.
- Service получает уже подготовленное значение id.
- Service возвращает товар или undefined.
- Controller решает, какой HTTP-ответ сформировать.

Кто знает о req и res

Route и Controller работают с HTTP, Service – с бизнес-логикой

Express.js
SIMPLE APIS
BIG POSSIBILITIES



HTTP / Express

Работа с HTTP-запросами и ответами



1. Route

Связан с HTTP-маршрутом

GET /api/products/:id

POST /api/products

...

```
// routes/products.js
router.get('/:id', getProductById)
router.post('/', createProduct)
```

Определяет endpoint
и передаёт запрос controller.

HTTP request
(попадает в
обработчик)



2. Controller

Получает req и res
(от Express)

req

res

```
// controllers/products.js
const id = Number(req.params.id);
const product = service.findById(id);
return res.json(product);
```

- 1 читает req.params, req.query, req.body
- 2 вызывает service
- 3 формирует res.status(...).json(...)

id, data, filters

(обычные
JS-значения)

result / product / data
(обычные значения)

HTTP response

(res.status(...).json(...))



Business logic

Бизнес-логика приложения



3. Service

Получает только обычные значения
(не знает о req и res)

```
// services/products.js
findById(id) {
  // ... бизнес-логика
  return product;
}
```

- ✓ Получает только обычные значения (id, data, filters и т. д.)
- ✓ Не знает о req и res
- ✓ Выполняет бизнес-логику

Service не получает req и res!



Клиент

(браузер / API client)



Главное правило:

req и res живут в HTTP / Express слое.
В Service передаются только обычные JS-значения.

Service для создания товара

```
export function addProduct(data) {  
  const product = {  
    id: products.length + 1,  
    ...data  
  };  
  
  products.push(product);  
  return product;  
}
```

Что важно

- Service получает обычный объект data.
- Функция формирует новый объект товара.
- Service изменяет источник данных.
- Созданный товар возвращается controller.

Controller создания товара

```
export function createProduct(req, res) {  
  const product = addProduct(req.body);  
  
  res.status(201).json(product);  
}
```

Что важно

- Controller получает данные из req.body.
- Создание товара передаётся в service.
- Service возвращает созданный объект.
- Controller устанавливает HTTP-статус 201.
- JSON-ответ отправляется клиенту.

POST /api/products через три слоя

- 1 Route выбирает createProduct** Запрос: POST /api/products. Код маршрута и controller: `router.post('/', createProduct); export function createProduct(req, res) { const product = addProduct(req.body); res.status(201).json(product); }`
- 2 Controller получает req.body** Controller извлекает данные нового товара из тела HTTP-запроса.
- 3 Service создаёт товар** Service выполняет операцию создания товара.
- 4 Service возвращает объект** Результат: `{ id: 6, name: 'SSD 1TB', price: 45000 }`
- 5 Controller отправляет статус 201 и JSON** Каждый слой выполняет только свою часть операции.



Учебный вопрос 5. Взаимодействие слоёв

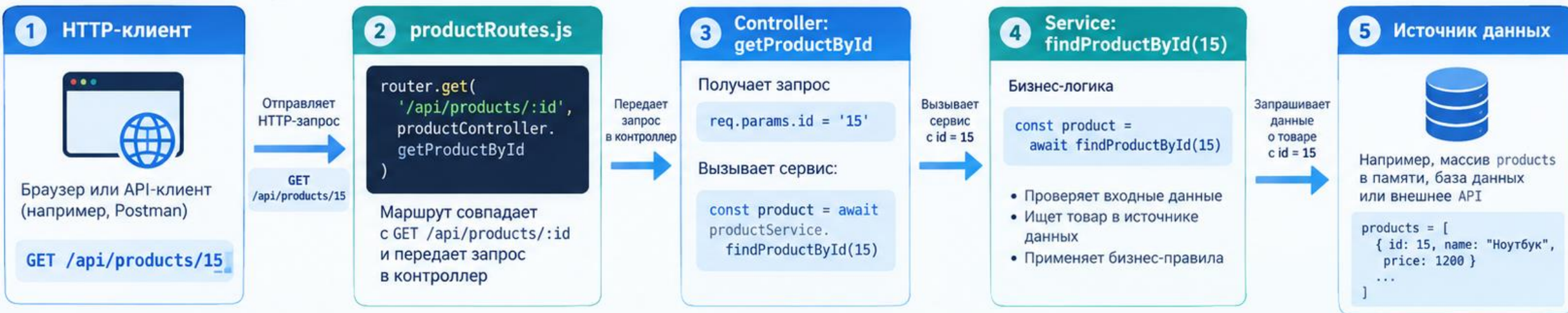
Полный путь запроса через routes, controllers и services

Полный путь HTTP-запроса через слои приложения

На примере: `GET /api/products/15`

ОТ ЗАПРОСА
ДО РЕЗУЛЬТАТА
ЧЕРЕЗ ВСЕ СЛОИ

1 Путь запроса (Request) — запрос проходит через слои приложения слева направо



2 Путь ответа (Response) — результат возвращается обратно через слои справа налево



Главная идея: HTTP-запрос проходит через слои приложения в определенном порядке, а результат возвращается обратно через контроллер к клиенту.

ПОНИМАЙ
СЛОИ — СОЗДАВАЙ
БОЛЬШЕ ВОЗМОЖНОСТЕЙ

Пример: GET /api/products/15

- 1 Route** `router.get('/:id', getProductById)` выбирает controller.
- 2 Controller** Получает `id` из `req.params` и преобразует его в `Number`.
- 3 Service** `findProductById(id)` выполняет поиск товара.
- 4 Controller** Получает результат `service` и выбирает статус ответа.
- 5 Response** Клиент получает JSON товара или сообщение 404.

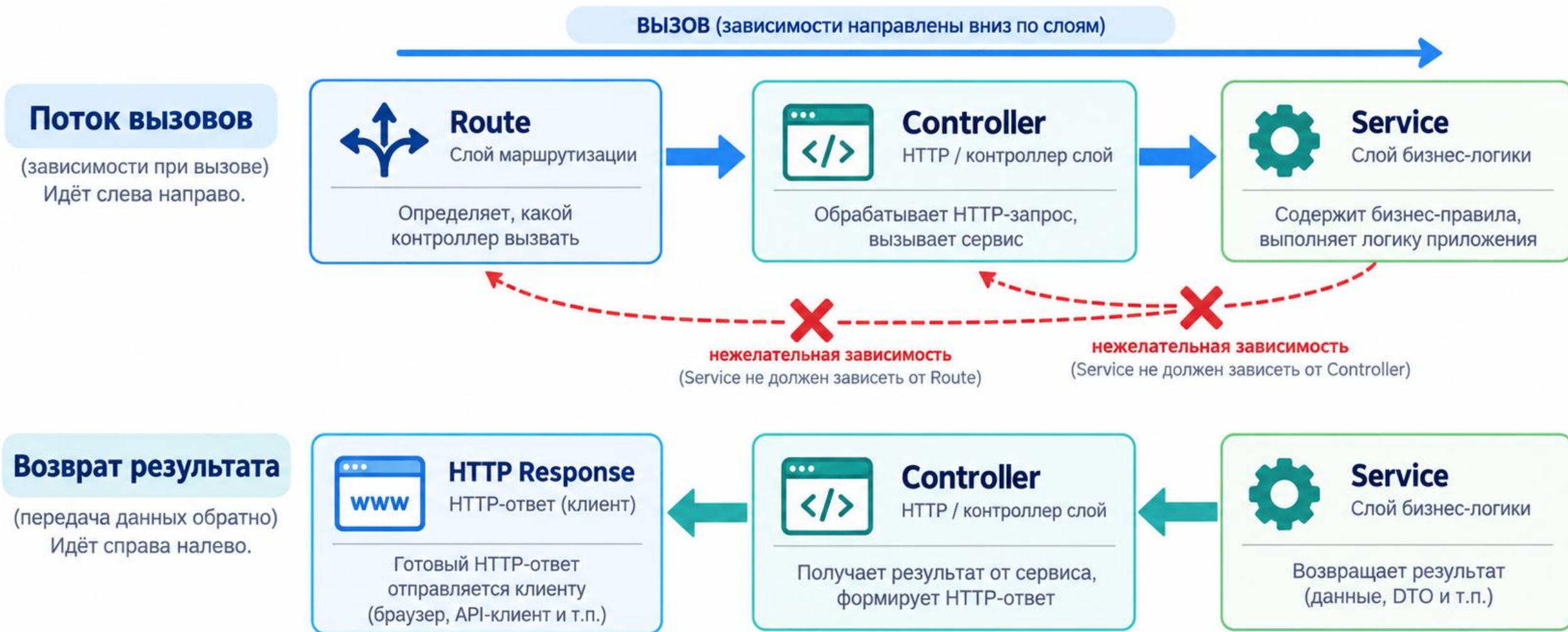
Направление зависимостей

- Route импортирует controller.
- Controller импортирует service.
- Service не импортирует route.
- Service не должен импортировать controller.
- Нижний слой не должен зависеть от деталей верхнего HTTP-слоя.

Главная идея

**Route → Controller → Service,
а результат возвращается в
обратном направлении.**

Направление зависимостей и возврат результата



Главная идея: зависимости направлены вниз по слоям, а результат возвращается обратно через controller к HTTP-ответу.

Где пока хранить данные

- На данном этапе курса товары могут оставаться обычным массивом JavaScript.
- Массив можно временно хранить внутри service или отдельного модуля данных.
- Главная цель занятия — не база данных, а разделение ответственности.
- В PO 4.3 service будет обращаться к PostgreSQL и ORM.
- Routes и controllers при этом потребуются менять значительно меньше.

Сегодня и после подключения базы данных

Сейчас

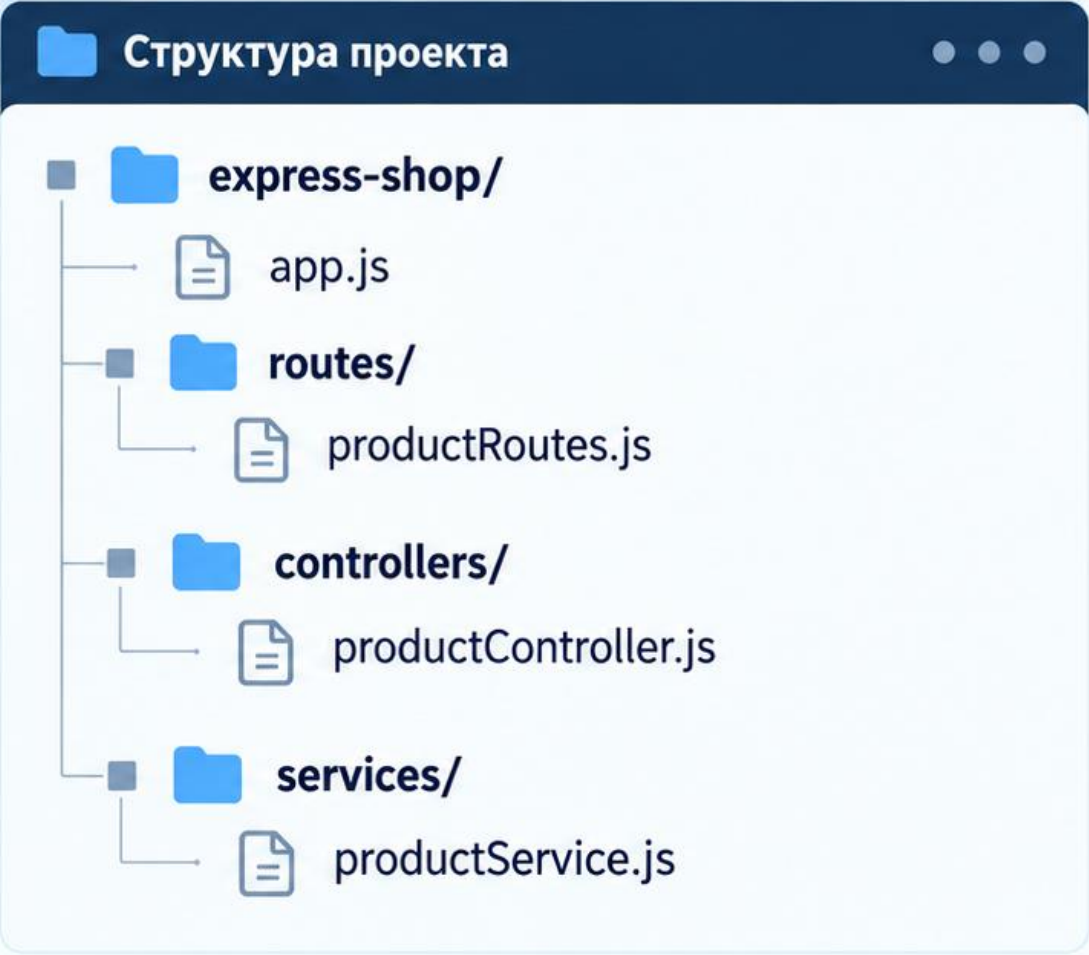
- Service работает с массивом
- `find()` ищет товар
- `push()` создаёт товар
- Данные исчезают после перезапуска

Позже

- Service обращается к PostgreSQL
- ORM выполняет запрос
- Данные сохраняются постоянно
- Route и controller почти не меняются

Структура проекта

```
express-shop/  
├── controllers/  
│   └── productController.js  
├── routes/  
│   └── productRoutes.js  
├── services/  
│   └── productService.js  
├── app.js  
├── package.json  
└── package-lock.json
```



Главная идея: зависимости направлены в одну сторону — от **app** к routes, затем к controllers и services.

Файлы Product-модуля

Файл	Что содержит
productRoutes.js	HTTP-методы и URL
productController.js	req, res и HTTP-статусы
productService.js	Операции с товарами и бизнес-логика
app.js	Middleware и подключение productRoutes

Связь app.js и Product Router

```
import productRoutes from './routes/productRoutes.js';  
  
app.use(express.json());  
app.use('/api/products', productRoutes);
```

Что важно

- app.js не импортирует controller напрямую.
- Главное приложение подключает Router.
- Router уже знает нужные controllers.
- Controllers знают нужные services.
- Так каждый уровень взаимодействует только с соседним уровнем.

Преимущества разделения

- Файлы становятся короче и имеют более понятное назначение.
- Проще находить код конкретной операции.
- Бизнес-логику можно использовать вне HTTP-обработчика.
- Services легче тестировать отдельно от Express.
- Замена способа хранения данных меньше влияет на остальные части приложения.
- Команде проще параллельно работать над разными слоями.

Не нужно создавать лишние слои заранее

- Архитектура должна помогать понимать приложение, а не усложнять его.
- Для маленького учебного проекта достаточно routes, controllers и services.
- Не следует создавать дополнительные абстракции без практической необходимости.
- Структуру можно расширять постепенно по мере появления новых задач.

Главная идея

Цель архитектуры — разделить ответственность, а не увеличить количество файлов.

Мини-практика

Разделите существующий Product Router на routes, controller и service.

Задание

- Оставить в `productRoutes.js` только endpoints
- Создать `productController.js`
- Перенести работу с `req` и `res` в controller
- Создать `productService.js`
- Перенести поиск, создание и изменение товаров в service
- Проверить GET, POST, PATCH и DELETE через HTTP-клиент

Критерии проверки

- Routes не содержат бизнес-логику
- Controller работает с `req` и `res`
- Service не использует `req` и `res`
- Существующие URL API не изменились
- Все маршруты продолжают работать

Контрольные вопросы

1. Зачем разделять Express-приложение на routes, controllers и services?
2. Являются ли controllers и services встроенными механизмами Express?
3. За что отвечает слой routes?
4. Что должен делать controller?
5. Почему controller знает об объектах req и res?
6. Какую задачу выполняет service?
7. Почему service не должен принимать req и res?
8. В каком направлении идут зависимости между слоями?
9. Что потребуется изменить в service после подключения PostgreSQL?
10. Почему разделение ответственности облегчает тестирование и сопровождение приложения?

Итоги занятия

1

Express не требует конкретной структуры проекта, но приложение удобно разделять по ответственности.

2

Routes описывают endpoints и связывают их с controllers.

3

Controllers работают с HTTP-запросом и HTTP-ответом.

4

Services выполняют бизнес-операции и не зависят от req и res.

5

Типичное направление вызовов: Route → Controller → Service.

6

Результат возвращается Service → Controller → Client.

7

Такое разделение уменьшает связанность кода и подготавливает приложение к подключению базы данных.